EX 4: IMPLEMENTATION OF A CONVOLUTIONAL NEURAL NETWORK (CNN)

AIM

To implement a Convolutional Neural Network (CNN) using PyTorch for image classification on the CIFAR-10 dataset, evaluate its performance on test images, and visualize the learned filters in the convolutional layers.

PROCEDURE

1. **Import the required libraries, including PyTorch, torchvision, and Matplotlib.**
2. **Load the CIFAR-10 training and testing datasets and normalize the image pixel values.**
3. **Define a CNN architecture consisting of convolutional layers, ReLU activation, max-pooling layers, and fully connected layers.**
4. **Initialize the model and select the appropriate computing device (CPU or GPU).**
5. **Define the cross-entropy loss function and Adam optimizer.**
6. **Train the CNN for 10 epochs using forward propagation, backpropagation, and weight updates.**
7. **Evaluate the trained model on the test dataset and calculate its classification accuracy.**
8. **Visualize the learned filters of the first convolutional layer to understand the features extracted by the network.**

CODE

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt


# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)


# Load and preprocess CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])
```

```
trainset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=64, shuffle=True
)

testset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

testloader = torch.utils.data.DataLoader(
    testset, batch_size=64, shuffle=False
)

classes = trainset.classes

# Define CNN architecture
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()

        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
```

```
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)

        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))

        x = x.view(-1, 32 * 8 * 8)

        x = self.relu(self.fc1(x))
        x = self.fc2(x)

        return x

# Initialize model, loss function, and optimizer
model = SimpleCNN().to(device)

lossfn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Train the CNN
epochs = 10
losses = []

for epoch in range(epochs):
    running_loss = 0.0
    model.train()

    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)
```

```
        optimizer.zero_grad()

        outputs = model(inputs)
        loss = lossfn(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(trainloader)
    losses.append(epoch_loss)

    print(f"Epoch {epoch + 1}, Loss: {epoch_loss:.4f}")

# Evaluate model performance
model.eval()
correct, total = 0, 0

with torch.no_grad():
    for images, labels in testloader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        _, predicted = outputs.max(1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f"Accuracy on test data: {accuracy:.2f}%")

# Visualize training loss
```

```
plt.plot(range(1, epochs + 1), losses, marker='o')
plt.title("CNN Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

# Visualize learned filters
def visualize_filters(layer, n_filters=8):
    filters = layer.weight.data.clone().cpu()

    fig, axs = plt.subplots(1, n_filters, figsize=(15, 4))

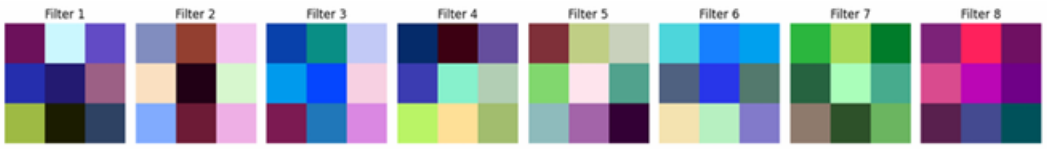
    for i in range(n_filters):
        f = filters[i]
        f = (f - f.min()) / (f.max() - f.min() + 1e-8)

        axs[i].imshow(f.permute(1, 2, 0))
        axs[i].axis('off')
        axs[i].set_title(f'Filter {i + 1}')

    plt.tight_layout()
    plt.show()

visualize_filters(model.conv1)
```

OUTPUT



RESULT

Thus, a Convolutional Neural Network (CNN) was successfully implemented using PyTorch and trained on the CIFAR-10 dataset. The model was evaluated using test accuracy, its training loss was visualized, and the learned filters of the first convolutional layer were displayed to understand feature extraction in image classification.